

Evaluación y Consistencia del Modelo UML

App de Recomendaciones de Eventos en la Ciudad

Nombre: Sofia Rodas -Estefania Restrepo

Ficha: 3115970



Objetivo de la Presentación

1 Evaluar la consistencia del modelo UML construido

Análisis exhaustivo para asegurar la robustez y fiabilidad del diseño del sistema.

2 Argumentar la coherencia entre requisitos, casos de uso, clases y actividades

Demostrar cómo cada componente del modelo se alinea con los demás para cumplir los objetivos del proyecto.

3 Identificar posibles inconsistencias y proponer mejoras

Detectar brechas o fallos en el diseño que puedan afectar la funcionalidad o el rendimiento de la aplicación.

Descripción General del Sistema

Nuestra aplicación de recomendación de eventos en la ciudad busca conectar a los usuarios con experiencias relevantes y facilitar la gestión para organizadores y administradores.

Funcionalidades Clave:

- Consultar eventos disponibles en la ciudad.
- Recibir recomendaciones personalizadas basadas en intereses.
- Registrar y administrar eventos para organizadores.
- Gestionar usuarios y eventos para administradores.

Actores Principales:

- **Usuario:** Busca y recibe recomendaciones.
- **Organizador:** Publica y gestiona eventos.
- **Administrador:** Supervisa y controla la plataforma.



Modelos Construidos para el Análisis

Para asegurar una representación completa y detallada del sistema, hemos desarrollado una serie de modelos UML clave.



Diagrama de Casos de Uso

Describe las interacciones entre los actores y el sistema, definiendo la funcionalidad externa.



Diagrama de Clases (Análisis)

Representa la estructura estática del sistema, mostrando clases, atributos y relaciones.



Diagrama de Actividades

Modela el flujo de control de los procesos de negocio, visualizando las acciones y decisiones.



Matriz de Trazabilidad

Documenta las relaciones entre los requisitos y otros elementos del modelo para asegurar la cobertura.

Estos modelos se complementan para ofrecer una visión integral desde diferentes perspectivas del sistema.

Consistencia: Requisitos Funcionales y Casos de Uso

La trazabilidad entre requisitos funcionales y casos de uso es fundamental para garantizar que todas las necesidades del usuario sean satisfechas por el sistema.



Requisito Funcional: Recomendar eventos personalizados

Caso de Uso Asociado: **Recibir recomendaciones**

Requisito Funcional: Registrar eventos

Caso de Uso Asociado: **Registrar evento**

Requisito Funcional: Consultar disponibilidad de eventos

Caso de Uso Asociado: **Buscar eventos**

Esta correspondencia uno a uno asegura una cobertura completa de los requisitos a nivel funcional.

Consistencia: Casos de Uso y Modelo de Clases

Cada funcionalidad descrita en los casos de uso encuentra su soporte estructural en las clases del sistema, garantizando que el diseño arquitectónico respalde el comportamiento deseado.

Ejemplos de Correlación:

Caso de Uso: Registrar Evento

Clases de Apoyo: **Organizador, Evento**

Caso de Uso: Guardar Evento en Favoritos

Clases de Apoyo: **Usuario, Evento, Favorito**

Caso de Uso: Recibir Recomendaciones

Clases de Apoyo: **Recomendacion, Usuario, Evento**



Esta estrecha relación asegura que no existan funcionalidades "en el aire" sin su respectivo respaldo en la estructura de datos del sistema.

Consistencia: Diagrama de Actividades

El diagrama de actividades para el proceso "Recibir recomendaciones" valida el flujo de negocio con los requisitos y el caso de uso correspondiente.



El flujo secuencial de actividades modelado en el diagrama se alinea perfectamente con la descripción detallada del caso de uso, desde la autenticación hasta la presentación de resultados, reflejando fielmente los requisitos iniciales.

Matriz de Trazabilidad: Cobertura Integral

La matriz de trazabilidad es una herramienta crucial para verificar la alineación entre todos los artefactos del proceso de diseño y desarrollo.

Requisito Funcional	Caso de Uso	Clases Involucradas	Actividad Asociada
RF001: Recomendar eventos	CU001: Recibir recomendaciones	Usuario, Evento, Recomendacion	Flujo de Recomendación
RF002: Registrar eventos	CU002: Registrar evento	Organizador, Evento	Proceso de Registro de Evento
RF003: Guardar eventos	CU003: Guardar en Favoritos	Usuario, Evento, Favorito	Gestión de Favoritos

Esta matriz garantiza que todos los requisitos estén representados, que no existan funcionalidades sin justificación y que cada clase tenga una relación clara con algún requisito, asegurando un diseño robusto y auditable.



Posibles Inconsistencias Detectadas

Durante el proceso de evaluación, se identificaron algunas áreas que requerían ajustes para fortalecer la consistencia y la integridad del modelo.



Omisión de Métodos para Aprobación

La clase **Evento** carecía de métodos explícitos para gestionar el ciclo de vida de aprobación de eventos por parte de un administrador.



Falta de Clase Intermedia para Favoritos

La relación entre **Usuario** y **Evento** para la funcionalidad de favoritos no estaba suficientemente modelada, necesitando una entidad propia.



Validación de Autenticación Incompleta

Ciertos flujos, como la recomendación de eventos, no incluían explícitamente la validación de la sesión del usuario.

Estas inconsistencias, aunque menores, podrían haber llevado a errores funcionales o de seguridad si no hubieran sido corregidas a tiempo.

Mejoras Realizadas en el Modelo

Tras la identificación de inconsistencias, se implementaron las siguientes mejoras clave para optimizar la robustez y funcionalidad del sistema.

01

Atributo "estado" en la clase Evento

Se añadió un atributo para gestionar el ciclo de vida de los eventos (e.g., Pendiente, Aprobado, Rechazado).

02

Incorporación de la clase Favorito

Se creó una clase intermedia para modelar la relación muchos a muchos entre **Usuario** y **Evento** para los favoritos.

03

Ajuste del flujo del diagrama de actividades

Se incluyó explícitamente un paso de "Validar Autenticación" antes de procesar recomendaciones o acciones sensibles.

04

Refuerzo de la relación entre Recomendacion, Usuario y Evento

Se clarificaron las asociaciones y cardinalidades para una generación de recomendaciones más precisa.

Estas correcciones aseguran un modelo UML más consistente, completo y alineado con los requisitos del sistema.

Argumentación Final: Consistencia del Modelo

La evaluación exhaustiva de los modelos UML confirma la consistencia y robustez de nuestro diseño, asentando una base sólida para el desarrollo del sistema.

Alineación de Requisitos y Funcionalidades

Cada requisito funcional se mapea directamente a un caso de uso, asegurando que toda necesidad del usuario esté cubierta por el sistema.

Soporte Estructural Sólido

La estructura de clases del sistema respalda cada comportamiento y funcionalidad definida, garantizando la viabilidad del diseño.

Procesos Claros y Coherentes

Los diagramas de actividades reflejan con precisión los flujos de negocio, validando la lógica operativa del sistema.

Cobertura Integral Validada

La matriz de trazabilidad asegura una interconexión y cobertura completa entre requisitos, casos de uso, clases y actividades.

Este nivel de consistencia minimiza riesgos, facilita el mantenimiento y asegura que el producto final cumpla con las expectativas.

Conclusión

La evaluación de los modelos UML nos ha permitido reafirmar la solidez, coherencia y alineación de nuestro diseño con los objetivos del proyecto.

Modelo Coherente y Estructurado

El modelo UML es consistente, bien estructurado y perfectamente alineado con todos los requisitos del sistema.

Diseño Fortalecido y Menos Errores

La retroalimentación cruzada ha fortalecido el diseño, permitiendo detectar y corregir errores de forma proactiva antes de la implementación.

Comprensión Integral del Sistema

El uso de UML facilita una comprensión holística del sistema desde múltiples perspectivas, mejorando la comunicación entre los equipos.

Este enfoque asegura un producto final robusto, funcional y fácil de mantener.

